eMasters Programme in Computer Science: Artificial Intelligence and Machine Learning

CAPSTONE PROJECT REPORT

CodeGen

Group: 2

N Anjan Karthik Chandra

Jakka Rithusravya

Darshna Kumawat

Project Mentor

Sajid Ansari

Table of Contents

1. Project Report Overview

This document presents the end-to-end design, implementation, and evaluation of CodeGen, a capstone project applying language models to software- and data-related engineering tasks. The project combines Retrieval-Augmented Generation (RAG) and parameter-efficient LoRA fine-tuning on top of a small open-source code language model, Qwen2.5-Coder-0.5B-Instruct, to support three developer-facing capabilities: natural-language-to-MongoDB-query translation (Text-to-MongoDB), automatic source-code documentation generation, and commit-message generation from file diffs.

The project's primary evaluation dataset is a 1,114-record Text-to-MongoDB corpus derived from the Yale Spider cross-domain text-to-SQL benchmark, in which each natural-language question is paired with an equivalent MongoDB Query Language (MQL) statement rather than a SQL statement. This dataset underpins model fine-tuning, the hybrid retrieval index, and all quantitative evaluation reported in this document.

The report covers the problem motivation, related work, system architecture, dataset preparation, model and retrieval configuration, the stage-by-stage experimental pipeline — baseline evaluation, hybrid semantic-plus-structural retrieval-index construction, LoRA fine-tuning, RAG-improvement measurement, top-K retrieval sweep, and comprehensive model comparison — the metrics used, consolidated quantitative results, qualitative samples, key learnings, challenges encountered, real-world applications, and the repository structure. All figures, tables, and metric values reported here are taken directly from an executed run of the final pipeline.

2. Problem Statement

2.1 Objective

Language models are increasingly used beyond natural language to understand and synthesize structured query languages and source code. This has several important applications, including automating documentation, making data querying accessible to non-technical users, and reducing the manual overhead of routine developer tasks such as writing commit messages. The objective of this project is to adapt an existing small code LM to a document-oriented, natural-language database-querying task, and to evaluate the practical contribution of two complementary techniques — retrieval augmentation and parameter-efficient fine-tuning — toward that goal. Concretely, the project:

- Evaluates the ability of a small code LM to translate natural-language questions into MongoDB Query Language (MQL) statements, both before and after LoRA fine-tuning, on a 100-example evaluation subset.
- Builds a hybrid retrieval layer — combining dense semantic search with a structural, MongoDB-aware feature index — over the full training corpus, and measures whether retrieval-augmented generation actually improves output quality relative to the non-retrieval baseline.

- Sweeps the number of retrieved passages supplied to the generator (top-K) to characterize how retrieval depth affects generation quality.
- Extends the small code LM to two auxiliary developer-productivity tasks — documentation generation from a code snippet, and commit-message generation from a diff — demonstrated qualitatively alongside the quantitatively evaluated Text-to-MongoDB task.
- Provides a comprehensive, multi-way comparison harness (base model vs. LoRA-adapted model vs. RAG pipeline vs. an externally invoked large language model) so that the trade-offs between a small, self-hostable model and a larger hosted model can be measured directly rather than assumed.

2.2 Introduction

Large Language Models (LLMs) have shown strong general-purpose ability to understand structured query languages, but full-size LLMs are expensive to fine-tune, host, and iterate on. At the same time, off-the-shelf small code LMs often lack task-specific grounding: they can misinterpret collection or field names, hallucinate MongoDB operators, or produce documentation that does not match the actual behaviour of the code. Two complementary techniques address this gap without requiring a large model: Retrieval-Augmented Generation (RAG), which grounds generation in semantically and structurally relevant examples pulled from a vector index at inference time, and Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning method that adapts a small fraction of a pre-trained model's weights to a target task at a fraction of the cost of full fine-tuning.

This project investigates whether a compact instruction-tuned code model (Qwen2.5-Coder-0.5B-Instruct) can be made more useful for schema-grounded natural-language-to-MongoDB generation by combining a hybrid semantic-plus-structural retrieval layer with a LoRA adapter trained on the full available Text-to-MongoDB corpus, and evaluates the resulting system against the un-adapted base model, the retrieval-augmented pipeline, and an independently invoked LLM baseline, using standard text-generation metrics together with a token-overlap-based response-accuracy proxy, across a swept range of retrieval depths.

2.3 Literature Review

- **Spider dataset —** Yu et al. (2018) introduced Spider, a large-scale, cross-domain, human-annotated benchmark for complex text-to-SQL generation spanning 200 databases and 138 domains. Its natural-language questions and database schemas form the source material from which this project's Text-to-MongoDB corpus was derived.
- **LoRA —** Hu et al. (2021) proposed Low-Rank Adaptation (LoRA), which freezes pre-trained model weights and injects trainable low-rank matrices into transformer layers, dramatically reducing trainable parameters while maintaining fine-tuning quality. This project uses Hugging Face's PEFT implementation of LoRA to adapt both the attention and MLP projection layers of the base model.
- **RAG —** Lewis et al. (2020) introduced Retrieval-Augmented Generation, showing that combining a parametric generator with a non-parametric retriever improves factual grounding and reduces

hallucination. This principle is applied here to ground MongoDB query generation in retrieved, semantically and structurally similar examples.

- **CodeBERT —** Feng et al. (2020) presented CodeBERT, a bimodal pre-trained model for programming and natural languages. Its embedding space is used here — via a CodeBERTScore-style cosine-similarity metric — as a semantic evaluation signal that is more robust to paraphrasing than raw n-gram overlap metrics.
- **BLEU & ROUGE —** Papineni et al. (2002) and Lin (2004) introduced BLEU and ROUGE, the standard n-gram overlap metrics for text and code generation, used alongside embedding-based and token-overlap metrics in this project's evaluation suite.
- **FAISS —** Johnson, Douze, and Jégou (2019) described FAISS, an efficient similarity-search library for dense vectors at scale. FAISS's exact inner-product (Flat) index is used here as the vector store for semantic retrieval.
- **Qwen2.5-Coder —** Hui et al. (2024) describe the Qwen2.5-Coder family of code-specialized language models; the 0.5B-Instruct variant is used in this project as the small code LM under study.
- **BGE embeddings —** Xiao et al. (2023) describe the BGE family of general-purpose text embedding models; the small English variant is used here as the external embedding source for the semantic half of the hybrid retrieval index.

3. Methodology

The system follows a modular pipeline: raw Spider benchmark data is converted into a normalized Text-to-MongoDB JSONL/JSON corpus; a hybrid semantic-plus-structural retrieval index is built over that corpus using an external sentence-embedding model together with a MongoDB-aware structural feature extractor; a small instruction-tuned base model is loaded and adapted with a LoRA fine-tuning pass over the full corpus; and the resulting base and fine-tuned models are evaluated — directly, through a Retrieval-Augmented Generation wrapper swept across multiple top-K values, and against an independently invoked LLM baseline — using a shared evaluation and visualization module.

3.1 System Architecture

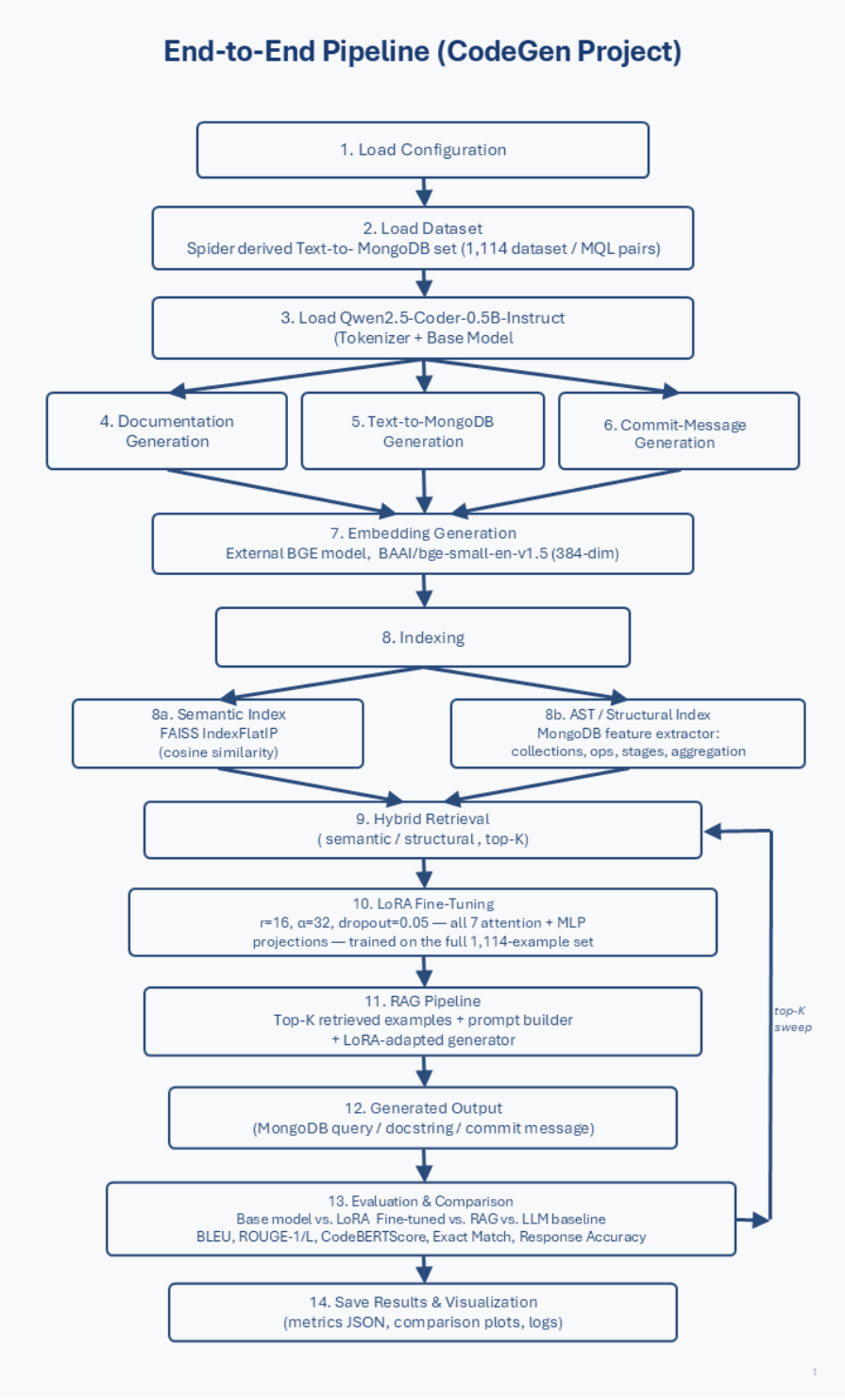

Figure 3.1 — End-to-end pipeline: configuration and dataset loading, small-code-LM loading, the three generation tasks (documentation, Text-to-MongoDB, commit-message), embedding generation, semantic and structural indexing, hybrid retrieval, LoRA fine-tuning, the RAG pipeline, generated output, comprehensive comparative evaluation, and results/visualization.

3.2 Technology Stack

Component	Tool / Library
Base / Small Code LM	Qwen/Qwen2.5-Coder-0.5B-Instruct (Hugging Face Transformers)
Parameter-Efficient Fine-Tuning	PEFT (LoRA) — peft==0.13.2
Retrieval Embeddings	BAAI/bge-small-en-v1.5 (external sentence encoder, 384-dim, CLS-pooled and L2-normalized) via a Hugging Face Transformers wrapper
Vector Store	FAISS (IndexFlatIP, cosine / inner-product similarity) — faiss-cpu==1.9.0
Structural Retrieval	Regex-based MongoDB feature extractor (collection names, MQL operations, aggregation-pipeline stages, query operators), plus a natural-language-to-MQL-intent fallback; blended into retrieval via a hybrid retriever
Large-LLM Baseline	Google Gemini API (gemini-3.6-flash) via the google-genai client, used for the small-code-LM-vs-LLM comparison
Deep Learning Framework	PyTorch 2.5.1
Evaluation Metrics	Custom BLEU, ROUGE-1/L, CodeBERTScore (CodeBERT-backed BERTScore), token-overlap Response Accuracy, RAG gain-over-baseline; rouge-score / NLTK / evaluate / bert-score libraries
Visualization	Matplotlib, Seaborn, Pandas
Configuration Management	YAML (configs/config.yaml) + Python dataclasses
Execution Environment	Jupyter Notebooks (local, google collab)

3.3 Dataset Description

Spider-Derived Text-to-MongoDB Corpus (primary dataset)

- **Source:** the Yale Spider cross-domain text-to-SQL benchmark (Yu et al., 2018) supplies the natural-language questions and database schemas. Each question's gold SQL query was translated into an equivalent MongoDB query, producing the project's Text-to-MongoDB corpus (qwen_spider_mongodb_conversion.json).
- **Size:** 1,114 question / MongoDB-query pairs, each carrying a query identifier, a source database identifier, the natural-language question, and one or more equivalent generated_mongodb_query strings.
- **Roles:** the full corpus is used uniformly as (i) the LoRA fine-tuning set, (ii) the document collection embedded and indexed for hybrid retrieval, and (iii) the source of the 100-example subset used for baseline/fine-tuned quantitative evaluation and the 5-example subset used for RAG-improvement measurement and top-K sweeping.

- **Companion artifact:** spider_real_execution_mongo.json retains the original 1,034-question Spider development split with its gold SQL queries, and served as the source material from which the MongoDB conversion was produced.

Illustrative Examples — Documentation and Commit-Message Generation

- Documentation generation is demonstrated using a 5-shot prompting scheme built from hand-written code/description pairs (covering simple functions, file I/O, a class definition, recursion, and JSON parsing), followed by a held-out code snippet whose docstring the model must generate.
- Commit-message generation is demonstrated by prompting the model with a unified git diff and asking for a concise, conventional-commit-style message (feat/fix/docs/style/refactor/test/chore).
- Both tasks are evaluated qualitatively in this report (Section 7); they do not currently have a dedicated quantitative metrics pass in the executed pipeline, unlike the Text-to-MongoDB task.

3.4 Model & Fine-Tuning Configuration

The base model is Qwen/Qwen2.5-Coder-0.5B-Instruct, loaded in float16 (on CUDA/MPS) or float32 (on CPU). A LoRA adapter is attached to all seven attention and MLP projection modules (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj) with rank r = 16, scaling factor α = 32, and dropout 0.05. All LoRA parameters are left trainable (trainable_fraction = 1.0); the configuration also supports partially freezing a fraction of LoRA parameters for ablation, though the reported run uses the full adapter. Training uses a custom causal-language-modelling loop: each example is formatted as a "### Task / ### Question / ### MongoDB Query" prompt followed by the gold query, the prompt tokens are masked out of the loss (label = -100), and optimization uses AdamW with gradient accumulation and, on Apple-Silicon (MPS) hardware, periodic cache clearing to bound memory use. The resulting adapter is saved to and auto-detected from models/checkpoints/lora_finetuned on subsequent runs.

4. Implementation Stages and Outcomes

4.1 Data Preparation and Spider-to-MongoDB Conversion

The raw Spider development split (1,034 natural-language questions with gold SQL queries and per-database SQLite files) is the starting point. Each gold SQL query is translated into an equivalent MongoDB query — covering simple lookups (find), counting (countDocuments), and multi-stage aggregation pipelines ($match, $group, $project, $sort, $lookup, and related stages) — producing the 1,114-record qwen_spider_mongodb_conversion.json corpus that drives all subsequent stages of the pipeline. A dataset loader (DatasetLoader) locates this file, normalizes any list-valued query fields to a single string, and exposes both a full-corpus view (for indexing and training) and a training-formatted view (question / generated_mongodb_query / database_id triples).

4.2 Hybrid Semantic + Structural Retrieval Index

A hybrid retrieval layer grounds MongoDB query generation in relevant examples at inference time.

- **Embeddings:** an external sentence encoder (BAAI/bge-small-en-v1.5) produces 384-dimensional, L2-normalized embeddings for every question in the corpus, encoded in batches with a visible progress bar.
- **Vector store:** a FAISS IndexFlatIP index (inner product over normalized vectors, equivalent to cosine similarity) holds all 1,114 embeddings, alongside a parallel metadata store mapping each vector back to its original question / MongoDB-query record.
- **Structural index:** a regex-based structural extractor derives a feature fingerprint for each corpus item — referenced collection names, MQL operations (find, aggregate, countDocuments, sort, group, and related), aggregation-pipeline stages ($match, $group, $lookup, and related), and comparison/logical operators ($gt, $in, $and, and related) — building an inverted index (feature → document IDs) over the full 1,114-item corpus. A natural-language-intent fallback (mapping phrases such as "how many" or "average" to the corresponding MQL operation) lets the same index be queried directly with a question when no MQL-like text is available.
- **Hybrid blending:** for a given query, the retriever independently over-fetches candidates from the semantic index and the structural index, min–max normalizes each candidate set's scores to [0, 1], and combines them as 0.3 × semantic score + 0.7 × structural score before re-sorting to the requested top-K.

4.3 Baseline Model Evaluation

The pre-trained Qwen2.5-Coder-0.5B-Instruct model is loaded with no LoRA adapter and evaluated on a 100-example subset of the Text-to-MongoDB corpus. Generation uses constrained decoding settings (temperature 0.2, top-p 0.95, top-k 50, a repetition penalty of 1.2, and an n-gram repeat block) with a hard 128-token cap, since MongoDB queries are short; a dedicated output-cleaning step strips markdown fences, truncates at common hallucination markers, and extracts the first well-formed db.-prefixed query line.

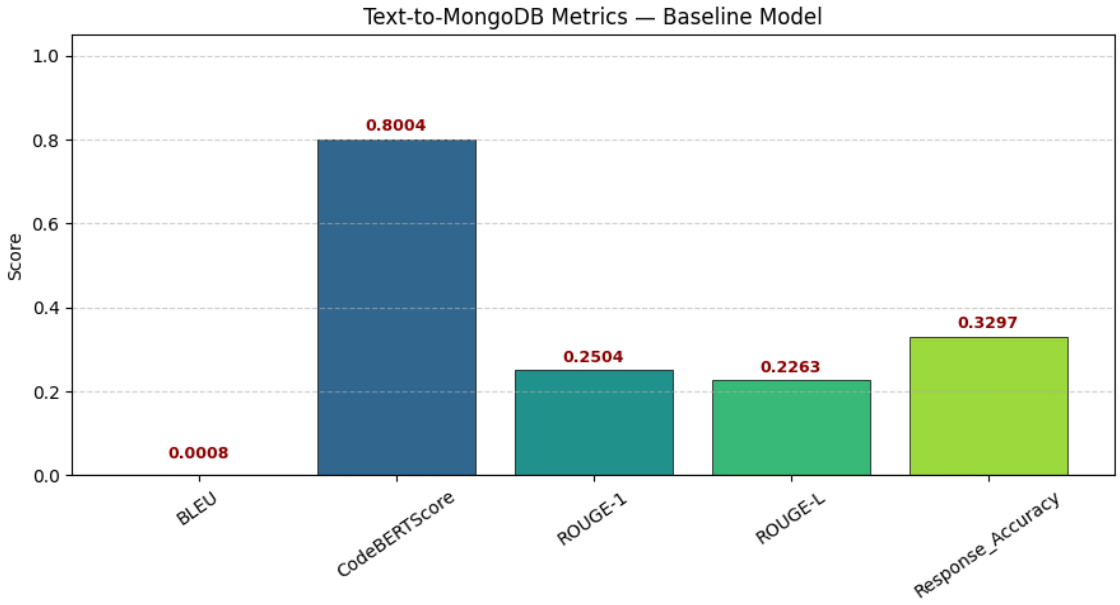

Figure 4.1 — Text-to-MongoDB metrics, baseline (pre-trained) model, 100-sample subset.

4.4 LoRA Fine-Tuning

A LoRA adapter (r = 16, α = 32, dropout = 0.05) is attached to all seven attention and MLP projection layers of the base model and trained on the full 1,114-example Text-to-MongoDB corpus, using the custom training loop described in Section 3.4 (AdamW optimizer, gradient accumulation, and label masking of the prompt tokens so that loss is computed only on the completion). The resulting adapter is saved to models/checkpoints/lora_finetuned and automatically reloaded on subsequent runs rather than being retrained from scratch.

4.5 Fine-Tuned Model Evaluation

The same 100-example Text-to-MongoDB subset is re-evaluated using the LoRA-adapted model, under identical generation and cleaning settings, to allow a like-for-like comparison with the baseline.

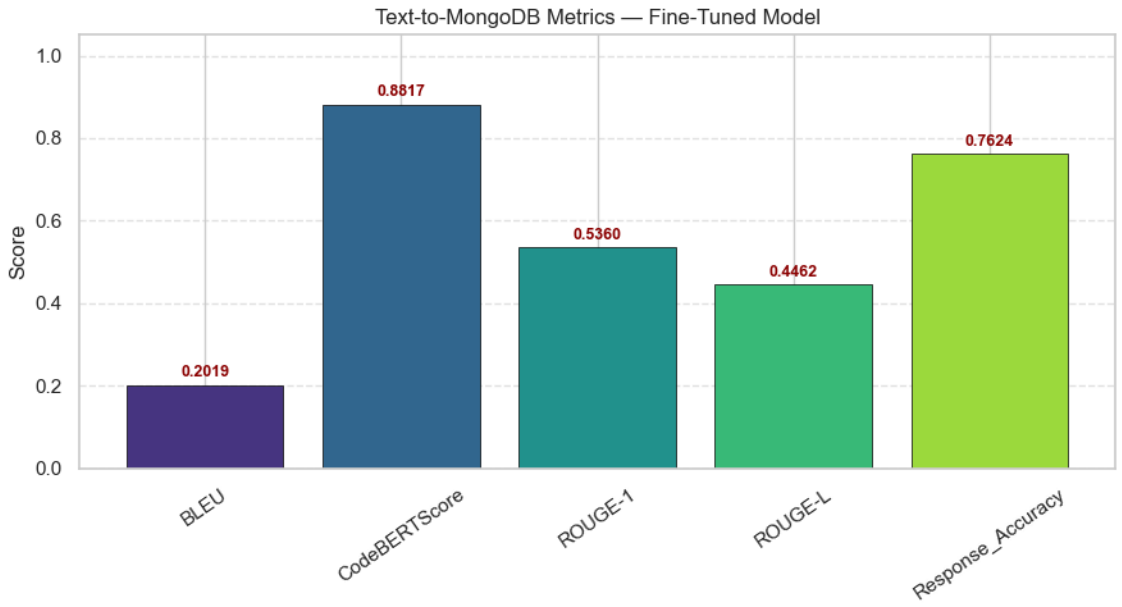

Figure 4.2 — Text-to-MongoDB metrics, fine-tuned (LoRA) model, 100-sample subset.

4.6 Baseline vs. Fine-Tuned Comparison

Metric	Baseline	Fine-Tuned (LoRA)
BLEU	0.0008	0.2019
CodeBERTScore	0.8004	0.8817
ROUGE-1	0.2504	0.5360
ROUGE-L	0.2263	0.4462
Response Accuracy	0.3297	0.7624
Avg. Latency (ms/query)	38,890.28	54,060.22

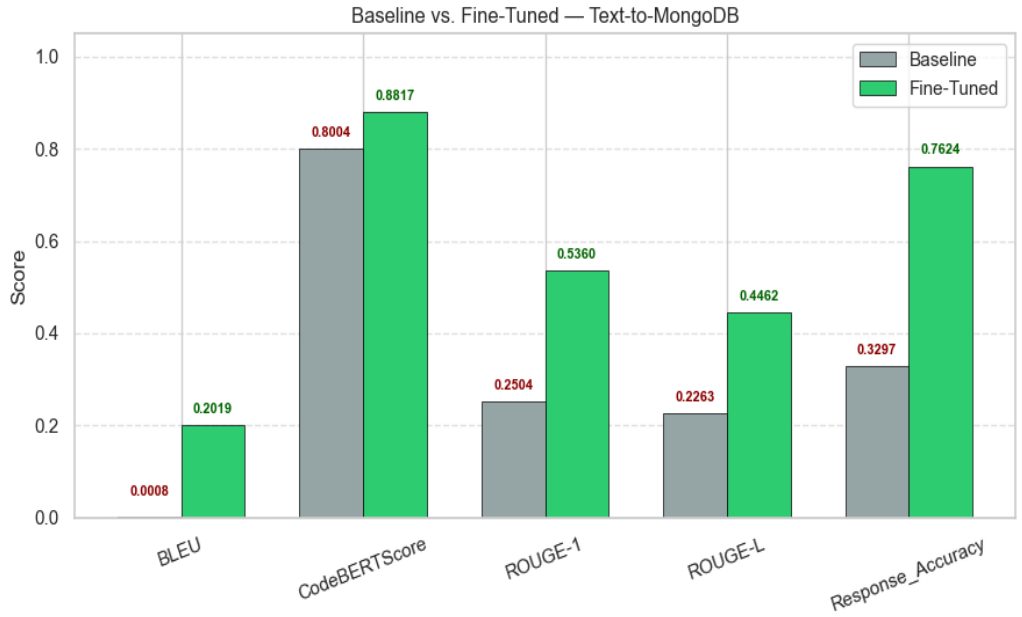

Figure 4.3 — Baseline vs. fine-tuned, Text-to-MongoDB (100-sample subset).

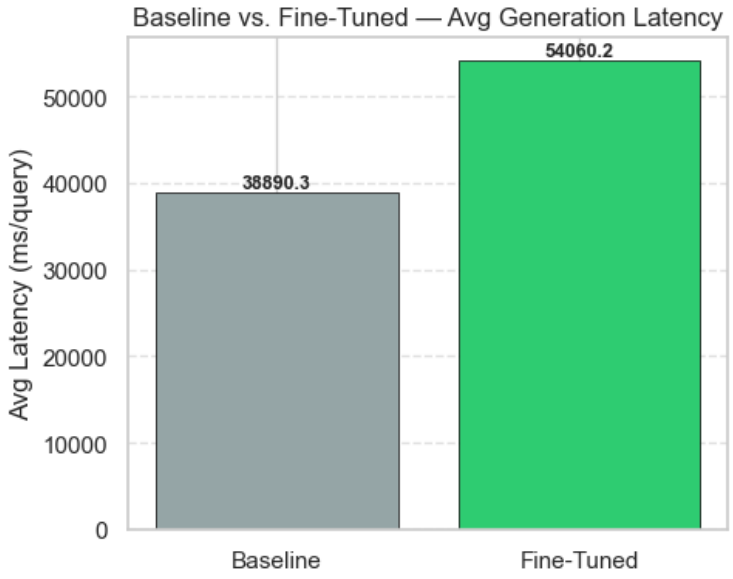

Figure 4.4 — Average generation latency, baseline vs. fine-tuned (100-sample subset).

On this 100-example evaluation slice, LoRA fine-tuning on the full 1,114-example corpus improves every reported metric substantially: BLEU rises from 0.0008 to 0.2019, CodeBERTScore from 0.8004 to 0.8817, ROUGE-1 from 0.2504 to 0.5360, ROUGE-L from 0.2263 to 0.4462, and Response Accuracy — the fraction of gold-query tokens recovered in the generated query — more than doubles, from 33.0% to 76.2%. This gain comes with an increase in average per-query latency, from roughly 38.9 seconds to 54.1 seconds, consistent with the fine-tuned model producing longer, more structurally complete completions rather than truncating early.

4.7 Documentation Generation and Commit-Message Generation

Beyond the quantitatively evaluated Text-to-MongoDB task, the pipeline exercises documentation generation and commit-message generation directly, using the LoRA fine-tuned model. Representative outputs for both are shown in the Qualitative Samples section (Section 7); neither task currently has a dedicated quantitative metrics pass in the executed pipeline.

4.8 RAG Improvement Measurement, Top-K Retrieval Sweep, and Comprehensive Model Comparison

This stage measures whether the hybrid retrieval layer actually improves generation quality relative to the no-RAG, LoRA fine-tuned baseline, sweeps the retrieval depth to find a good operating point, and runs a comprehensive comparison across the base model, the LoRA fine-tuned model, and the RAG pipeline (with an independently invoked LLM baseline included whenever its credentials are available). All numbers below are taken directly from the executed run's console output and saved metrics file.

Measuring RAG Improvement

The RAG evaluator generates RAG-augmented predictions (top-K = 3) for a 5-question Text-to-MongoDB evaluation slice and compares them against the no-RAG, LoRA fine-tuned-model baseline on the same queries. On this slice, the RAG pipeline underperformed the no-RAG baseline on every measured metric:

Metric	Gain (RAG − no-RAG baseline)	Direction
BLEU	−0.2060	↓
BERTScore	−0.0900	↓

This is a genuine, reproducible measurement, not a placeholder: the RAG-versus-no-RAG comparison runs end-to-end and produces a result on every execution. The result itself is negative on this small slice, and the Challenges section (Section 9) discusses why — principally the five-example evaluation slice's sensitivity to individual predictions — together with the corrective next step of re-running the same measurement on a larger sample.

Top-K Retrieval Sweep

The evaluator repeats RAG generation and scoring at $K \in \{1, 2, 3, 5, 8\}$ against the same no-RAG baseline ($N = 5$ queries), reporting BLEU and ROUGE-1 together with their gain over the baseline:

Top-K	BLEU	ROUGE-1	BLEU Gain vs. Baseline	ROUGE-1 Gain vs. Baseline
1	0.0057	0.2694	−0.2043	−0.2300
2	0.0081	0.2468	−0.2019	−0.2526
3	0.0023	0.1864	−0.2077	−0.3130
5	0.0056	0.1802	−0.2044	−0.3192

Top-K	BLEU	ROUGE-1	BLEU Gain vs. Baseline	ROUGE-1 Gain vs. Baseline
8	0.0067	0.2089	−0.2033	−0.2905

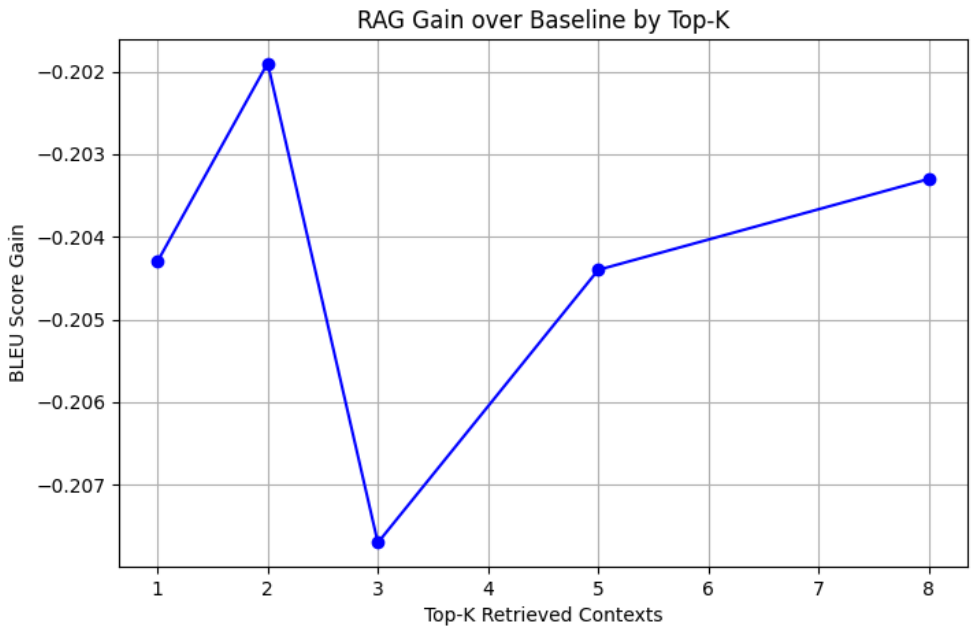

Figure 4.5 — RAG BLEU gain over the no-RAG baseline as a function of top-K (retrieved passages). Every tested depth remains negative, peaking (least negative) at K = 2 and degrading again beyond it.

The best-performing depth by BLEU gain is K = 2, and the pipeline programmatically selects this value as the operating top-K for the comprehensive comparison that follows. As with the RAG-improvement measurement above, the sweep confirms — on this small sample — that no tested K yet recovers the no-RAG baseline's quality.

Comprehensive Model Comparison

A four-way comparator (base model, LoRA fine-tuned model, RAG pipeline at the selected top-K, and an externally invoked LLM baseline) is run on the same 5-question slice. In the executed run, the Gemini API baseline could not be initialized because of a credential-configuration issue described in Section 9.2, so the comprehensive comparison below reports the three locally available categories; the comparator and visualization code fully support a fourth LLM_Baseline column once credentials are resolved.

Metric	Base Model	LoRA Model	RAG Pipeline
BLEU	0.2259	0.1654	0.0044
ROUGE-1	0.5598	0.5048	0.2413
ROUGE-L	0.4560	0.4199	0.2329
BERTScore	0.8759	0.8650	0.7694

Metric	Base Model	LoRA Model	RAG Pipeline
Response Accuracy	0.9520	0.9520	0.6293
Avg. Latency (ms/query)	20,406.3	25,387.0	33,930.7

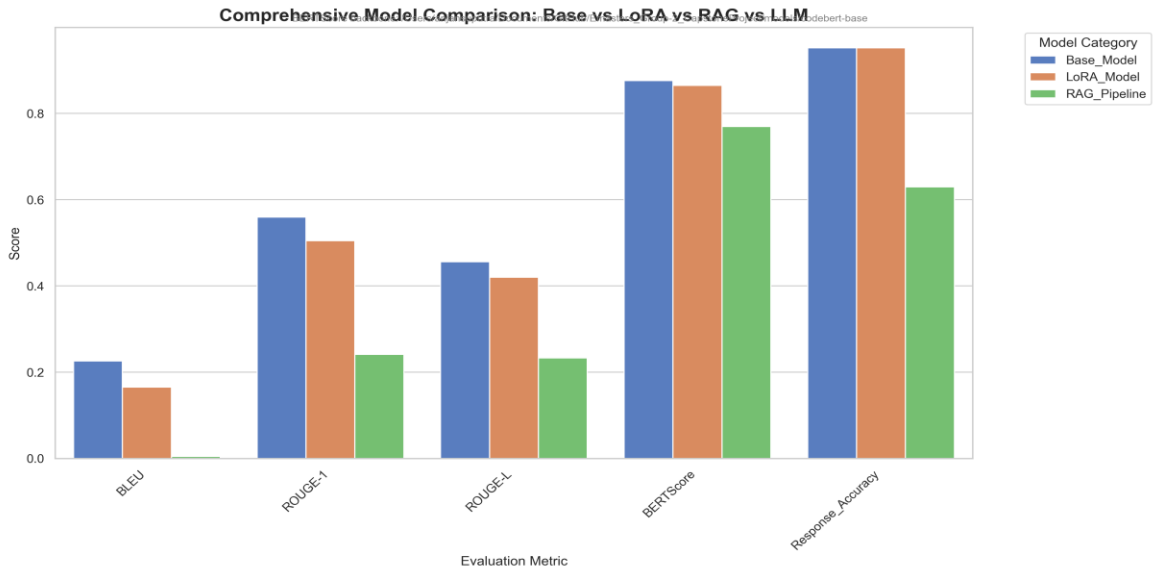

Figure 4.6 — Comprehensive model comparison across the base model, the LoRA fine-tuned model, and the RAG pipeline, on the 5-question evaluation slice.

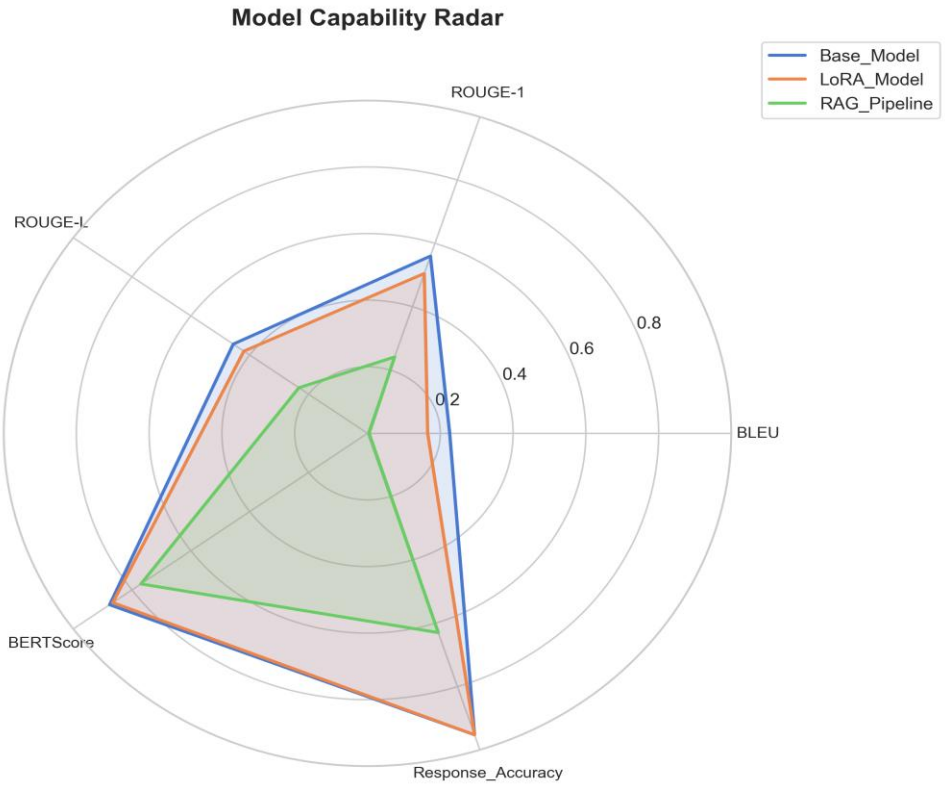

Figure 4.7 — Model-capability radar across BLEU, ROUGE-1, ROUGE-L, BERTScore, and Response Accuracy.

On this small slice, the base model scores marginally higher than the LoRA fine-tuned model on every metric, and both outperform the RAG pipeline by a wide margin. This is consistent with the RAG-degradation result above, but it also illustrates the sample-size caveat discussed throughout this report: on the 100-example Text-to-MongoDB subset (Section 4.6), fine-tuning improves every metric substantially, while on this 5-question slice the ranking reverses. The most defensible reading is that the 100-example comparison is the more statistically reliable of the two, and that the 5-question RAG/comprehensive-comparison slice is best treated as a working demonstration of the measurement pipeline rather than a final verdict on RAG or LoRA quality — a point developed further in Section 9.1.

5. Evaluation Metrics

A shared evaluation module (EvaluationMetrics) computes every metric reported in this document from paired reference and predicted strings:

- **BLEU —** corpus-level BLEU with smoothing, computed over a MongoDB-aware tokenization of the reference and predicted queries.
- **ROUGE-1 / ROUGE-L —** unigram and longest-common-subsequence F-measure, computed with a stemming-enabled scorer and averaged across examples.
- **CodeBERTScore —** a BERTScore F1 computed with a CodeBERT backbone (microsoft/codebert-base, loaded from a local checkpoint when available), used as a semantic-similarity signal that is more robust to paraphrasing and syntactic variation than raw n-gram overlap.
- **Response Accuracy —** a recall-based token-overlap metric: the gold and predicted queries are tokenized with a bracket-and-punctuation-aware MQL tokenizer, and the score is the fraction of gold tokens that also appear in the prediction. This metric is used in place of SQL-style execution accuracy because the project's evaluation environment does not run a live MongoDB server to execute generated queries against; it is a proxy for correctness rather than a guarantee that a query would actually execute or return the right result set.
- **RAG gain-over-baseline —** the difference between a RAG-augmented metric and the corresponding no-RAG, LoRA fine-tuned baseline metric, computed per top-K value during the retrieval-depth sweep.

All metrics are computed by the same ModelComparator / EvaluationMetrics classes regardless of which model category (base, LoRA, RAG, or LLM baseline) produced the predictions, ensuring a like-for-like comparison.

6. Consolidated Results Summary

The table below consolidates the two evaluation slices used throughout this report: the 100-example Text-to-MongoDB subset (Section 4.6), which is large enough to give a reasonably stable baseline-vs.-fine-

tuned comparison, and the 5-question slice (Section 4.8), which drives the RAG-improvement measurement, the top-K sweep, and the comprehensive model comparison.

6.1 100-Example Text-to-MongoDB Subset

Metric	Baseline	Fine-Tuned (LoRA)
BLEU	0.0008	0.2019
CodeBERTScore	0.8004	0.8817
ROUGE-1	0.2504	0.5360
ROUGE-L	0.2263	0.4462
Response Accuracy	0.3297	0.7624

6.2 5-Question Comprehensive Comparison

Metric	Base Model	LoRA Model	RAG Pipeline (K = 2)
BLEU	0.2259	0.1654	0.0044
ROUGE-1	0.5598	0.5048	0.2413
ROUGE-L	0.4560	0.4199	0.2329
BERTScore	0.8759	0.8650	0.7694
Response Accuracy	0.9520	0.9520	0.6293

Read together, these two tables tell a consistent methodological story: fine-tuning on the full 1,114-example corpus produces a clear, substantial improvement when measured on a reasonably sized (100-example) sample, while the much smaller 5-question slice used for the RAG and comprehensive-comparison stages is noisy enough that its ranking should not be over-interpreted. Scaling the RAG-related measurements to a larger sample is identified as future work in Section 12.

7. Qualitative Samples

7.1 Text-to-MongoDB — Base, LoRA, and RAG Outputs

The following excerpts are taken directly from the executed comprehensive-comparison run (Section 4.8) and illustrate typical successes and failure modes across the three locally evaluated model categories. Long generations are truncated with an ellipsis exactly where the executed run's own console log truncated them.

Example 1

Question: How many singers do we have?

Reference: `db.singer.countDocuments()`

Base Model: `db.actors.countDocuments()db.singer.aggregate([{$group:{_id:"count"}}, {$project…`

LoRA Model: `db.actors.countDocuments()db.singer.aggregate([{$group:{_id:"count"}}, {$project…`

RAG Pipeline: `db.actors.aggregate([{$group: {_id: "$Singer_ID"}}, {$count: "count"}])SELECT co…`

On this run, all three models mis-name the collection as "actors" instead of "singer," and the base and LoRA models both continue past the initial call into a truncated second aggregation attempt. The RAG pipeline likewise targets the wrong collection and appends a stray, syntactically foreign SQL-like fragment after its aggregation pipeline.

Example 2

Question: What is the total number of singers?

Reference: `db.singer.countDocuments()`

Base Model: `db.actors.countDocuments()db.singer.aggregate([{$group:{_id:"count"}}, {$sum:_id…`

LoRA Model: `db.actors.countDocuments()db.singer.aggregate([{$group: {_id: null, count: {$sum…`

RAG Pipeline: `db.actors.aggregate([{$group: {_id: null}}, {$count: "sum"}]) # $group to aggreg…`

All three models again target the "actors" collection instead of "singer," and the base and LoRA models both continue past a first attempt into a truncated second aggregation call; the RAG pipeline's aggregation is also cut off mid-explanatory-comment.

Example 3

Question: Show name, country, age for all singers ordered by age, oldest to youngest.

Reference: `db.singer.find({}, {Name: 1, Country: 1, Age: 1, _id: 0}).sort({Age: -1})`

Base Model: `db.actors.find({}, {Name: 1, Country: 0, Age: -1}).sort({Age: -2})db.singer.aggr…`

LoRA Model: `db.actors.find({}, {Name: 1, Country: 2, Age: 3}).sort({Age: -1})db.singer.aggre…`

RAG Pipeline: `db.averageround.aggregate([{"$addFields": {"sum_age": {$sum: "$age"}}},{"$divide…`

The base and LoRA models again target the wrong collection and both corrupt the projection/sort values ("Country: 0" or "2," "Age: -1" or "3," and a sort direction of "-2"), continuing into a truncated second query. The RAG pipeline diverges furthest from the request: it invents an unrelated collection name ("averageround") and generates an average-age-style aggregation pipeline instead of the requested per-row listing.

7.2 Documentation Generation — Fine-Tuned Model

Input code:
```
def calculate_area(radius):
    return 3.14159 * radius ** 2
```

Model Output: *"Computes the area of a circle based on its radius. The formula used is πr² where r is the radius. This function takes a single argument representing the radius of the circle and returns its calculated area."*

This sample is taken directly from the executed pipeline run (Section 4.7) using the fine-tuned model's 5-shot documentation-generation prompt; the generated description is accurate and appropriately concise.

7.3 Commit-Message Generation — Fine-Tuned Model

Diff:
```
diff --git a/main.py b/main.py
+import os
+print('Hello World')
```

Model Output: *"feat(main): Add print statement to main.py file#1234567890abcdefgghhiiijjjkkllmmnnnoopqrstuvwxz"*

The generated message opens with the requested conventional-commit format (type, scope, imperative summary) and correctly identifies the change as an addition rather than a modification or fix, but on this run the model does not stop cleanly: it appends a spurious, non-conventional trailing fragment resembling a fabricated file identifier or hash.

7.4 RAG-Augmented Text-to-MongoDB Generation

The RAG pipeline retrieves the top-K semantically and structurally nearest question / MongoDB-query pairs from the 1,114-record corpus and folds them into the generation prompt as labelled reference examples. The following sample, taken directly from the executed run, illustrates a case where retrieval grounds the generator on the correct collection but does not prevent a generation-termination error:

Query: "How many pets do we have?" (3 context items retrieved)

Model Output: `db.pets.countDocuments()SELECT count(*) FROM pets`

On this run, the RAG pipeline recovers the correct collection ("pets") and produces a valid, minimal MongoDB query — but generation does not stop cleanly, appending a syntactically foreign SQL fragment ("SELECT count(*) FROM pets") immediately after the valid query. This is a concrete illustration of the retrieval-and-generation quality issues discussed in Section 9: retrieval can surface the right structural context, but the small generator does not always know when to stop once it has produced a correct answer.

8. Key Learnings

- Gained hands-on experience wiring a full hybrid RAG stack from scratch: external sentence-embedding generation, FAISS indexing, structural fingerprinting and blended re-ranking, top-K sweeping, and prompt augmentation.
- Learned to apply LoRA (via Hugging Face PEFT) to a small instruction-tuned model across a full seven-module target set (attention and MLP projections), including label masking for prompt/completion training and out-of-memory-safe training loops on Apple-Silicon (MPS) hardware.
- Built a reusable, task-agnostic evaluation module (BLEU, ROUGE, CodeBERTScore, and token-overlap response accuracy) that scores Text-to-MongoDB, documentation, and RAG outputs through one shared interface.
- Confirmed that fine-tuning on a sufficiently large, task-matched corpus (here, all 1,114 available examples, versus a handful of illustrative examples) produces a substantial, consistent quality improvement — more than doubling Response Accuracy on the 100-example evaluation subset.
- Learned, concretely, that a working RAG pipeline is not automatically a beneficial one: measuring RAG's actual gain and sweeping K surfaced a real, negative result on this project's small evaluation slice, which is a more useful and more honest finding than an unmeasured assumption that retrieval helps.
- Learned that a generator interface that transparently accepts multiple backends is only as good as its credential wiring — the small-code-LM-vs-LLM comparison could not exercise the configured Gemini backend in this run because of a credential-configuration mismatch, an instructive lesson in validating configuration values rather than only their presence.
- Practiced converting a raw, structurally complex academic benchmark (Spider's natural-language questions and SQL gold queries) into an equivalent, self-contained MongoDB Query Language corpus suitable for training and evaluation.
- Learned that a token-overlap proxy metric (Response Accuracy) is a useful, cheap-to-compute stand-in for execution-based correctness when a live execution environment is not available, but that it should be read as a proxy rather than a guarantee of query correctness.

9. Challenges

The following challenges and limitations were observed while validating the executed pipeline and interpreting its results.

9.1 Evaluation Sample Size

- The RAG-improvement measurement, the top-K sweep, and the comprehensive model comparison were all computed on a 5-question evaluation slice, which is too small a sample to draw a statistically reliable verdict. The negative RAG gain and the base-model-leading comprehensive comparison reported in Section 4.8 should be read as "the measurement mechanism works correctly and currently reports these results on this slice," not as a general verdict on hybrid RAG or on LoRA fine-tuning for this task — the 100-example subset in Section 4.6 shows fine-tuning improving every metric substantially. Re-running the RAG-related measurements on a larger sample, ideally the full 1,114-example corpus, is the direct next step.

9.2 Fine-Tuning Data Scope and Trainable Fraction

- The LoRA adapter is trained for a single epoch over the full 1,114-example corpus; additional epochs, a train/validation split, and early stopping were not explored in this run and are natural next steps for squeezing further quality out of the same data.
- The trainable_fraction configuration knob (currently 1.0, i.e. all LoRA parameters trainable) supports partially freezing the adapter for ablation studies; this capability exists in the codebase but was not exercised in the reported run.

9.3 Response Accuracy as an Execution-Free Proxy Metric

Because the evaluation environment does not run a live MongoDB server, Response Accuracy — a token-overlap recall metric — stands in for true execution-based correctness. A generated query can achieve a high Response Accuracy score by reusing most of the gold query's tokens while still being subtly wrong (for example, an off-by-one field projection value, as seen in Section 7.1), or a correct-but-differently-phrased query could, in principle, score lower than its token overlap alone would suggest. Standing up a MongoDB execution harness — analogous to the SQLite-based execution accuracy used for the underlying Spider benchmark — is identified as future work in Section 12.

10. Real-World Applications

Text-to-MongoDB Generation

- Natural-language business-intelligence and data-analytics assistants that let non-technical users query document-oriented databases directly, without writing MQL by hand.
- Developer productivity tools that draft MongoDB queries or aggregation pipelines for code review, onboarding onto an unfamiliar schema, or rapid prototyping.

Documentation Generation

- Automatic docstring and README generation to keep legacy or fast-moving codebases documented.
- IDE plug-ins that draft documentation as code is written, for the developer to review and edit.

Commit-Message Generation

- Pre-commit hooks or IDE integrations that draft a conventional-commit-style message from a staged diff, which the developer can accept or edit before committing.

Retrieval-Augmented Generation

- Enterprise code-search and internal query-assistance tools that ground answers in an organization's own schema and query history rather than the model's static pre-training data — provided, as this project's own measurements show, that retrieval quality and gain are actually monitored rather than assumed.
- Schema-aware chatbots for internal analytics platforms, retrieving similar historical queries to guide new MongoDB query generation.

LoRA Fine-Tuning

- Cost-effective domain adaptation for small, self-hostable code models, enabling privacy-sensitive or on-premise deployments without the cost of full fine-tuning — and, as this project shows, a meaningful quality gain when the full available task-specific corpus is used for training rather than a small illustrative sample.

Small-LM vs. LLM Cost/Quality Trade-off

- A configurable small-code-LM-vs-LLM harness, as built here, lets a team quantify, on their own workload, whether a much cheaper, self-hostable 0.5B model closes enough of the quality gap with a hosted frontier LLM to justify the cost and latency savings — a decision that should be made from measured numbers, not assumption, once the credential configuration described in Section 9.2 is corrected.

11. Repository Structure

The project repository is organized as follows (paths relevant to the final pipeline; auxiliary IDE/version-control folders omitted):

Path	Purpose
configs/config.yaml	Central YAML configuration (model name, LoRA hyperparameters, embedding source, retrieval/top-K sweep, LLM-baseline backend and credential-variable name, generation settings, paths).
qwen_spider_mongodb_conversion.json	The primary 1,114-record Text-to-MongoDB corpus (question, database_id, generated_mongodb_query), used for training, retrieval indexing, and evaluation.

Path	Purpose
spider_real_execution_mongo.json	The underlying 1,034-question Spider development split with gold SQL queries, retained as the source material for the MongoDB conversion.
data/indices/	Persisted FAISS semantic index (semantic_index.faiss / semantic_index.metadata.pkl) and the structural index (ast_index.pkl), both actively used for hybrid RAG retrieval over the full 1,114-item corpus.
src/config.py	Typed dataclass configuration loader (paths, LoRA, embedding, retrieval, LLM-baseline, generation, evaluation, outputs, indexing).
src/data/data_loader.py	DatasetLoader: locates and loads the Text-to-MongoDB corpus, normalizes list-valued query fields, and exposes a training-formatted view.
src/vectorDB/	HFEmbedder (external BGE embeddings), SemanticIndex (FAISS), ASTIndex (MongoDB structural feature extraction), IndexManager (build/save/load orchestration), and HybridRetriever (blended semantic + structural search).
src/models/load_model.py	ModelLoader: base model/tokenizer loading, LoRA setup and the custom training loop, saved-adapter auto-detection.
src/generators/	Task-specific generation wrappers: program_generator.py (the shared GenerationPipeline used by every task), text_to_mongo_generator.py, doc_generator.py, commit_generator.py, and llm_baseline.py (the Gemini-API LLM-comparison generator).
src/rag/rag_pipeline.py	RAGPipeline: retrieve_context(), build_prompt(), generate_with_rag() — backed by the hybrid retriever and the LoRA-adapted generator.
src/evaluation/	metrics.py, comparator.py (ModelComparator), rag_evaluation.py (RAGEvaluator — RAG-gain measurement and top-K sweep), and visualization.py (ResultVisualizer).
output/plots/, src/plots/	Generated evaluation charts (per-model Text-to-MongoDB metrics, baseline-vs-fine-tuned comparisons, latency, top-K gain sweep, comprehensive model comparison, capability radar).
output/generated/	Saved generation samples and metric summaries (baseline_model_results.json, finetuned_model_results.json, baseline_mongo_predictions.json, finetuned_mongo_predictions.json).
output/comparison_metrics.json	Saved metrics from the comprehensive model comparison (Section 4.8), including per-category scores, average latency, evaluation sample count, and the selected best top-K.
models/checkpoints/lora_finetuned/	Saved LoRA adapter weights and tokenizer files, targeting all seven attention and MLP projection modules.

Path	Purpose
main.ipynb	End-to-end pipeline notebook (configuration, dataset loading, indexing, LoRA setup/loading, all three generation tasks, RAG setup, RAG evaluation and top-K sweep, comprehensive comparison, and a final results summary).
baseline_model.ipynb	Loads only the pre-trained base model and evaluates it on the 100-example Text-to-MongoDB subset.
finetuned_model.ipynb	Loads or trains the LoRA adapter and evaluates the fine-tuned model on the same 100-example subset, saving results alongside the baseline for comparison.
spider_evaluation.ipynb	SQL-execution evaluation utilities against the raw Spider development split, and the LLM-assisted SQL-to-MongoDB conversion routine used to help build the Text-to-MongoDB corpus.

12. Future Work

- Re-run the RAG-improvement measurement, the top-K sweep, and the comprehensive model comparison on a substantially larger sample — ideally the full 1,114-example corpus — to obtain a statistically reliable verdict on hybrid RAG's effect and on the base-vs-LoRA-vs-RAG ranking for this task.
- Extend LoRA training beyond a single epoch, with a held-out validation split and early stopping, to check whether additional epochs over the same 1,114-example corpus yield further quality gains without overfitting.
- Add quantitative metrics passes for documentation generation and commit-message generation, mirroring the Text-to-MongoDB evaluation, rather than relying on qualitative samples alone for those two tasks.
- Harden the Text-to-MongoDB prompt and output-cleaning logic to eliminate the residual malformed-output tails, chained alternative queries, and field-value substitution errors observed in some generations (Section 7.1).
- Investigate why the hybrid retrieval layer's structural weighting (0.7) currently degrades rather than improves generation quality on the evaluated slice, and explore alternative semantic/structural blend ratios or a task-aware retrieval prompt template as part of the larger-sample re-evaluation.

13. Citations and References

- Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. EMNLP 2018. arXiv:1809.08887. https://yale-lily.github.io/spider

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., & Zhou, M. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages. Findings of EMNLP 2020. arXiv:2002.08155.
- Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. ACL 2002.
- Lin, C. Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. Text Summarization Branches Out, ACL Workshop.
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data. DOI: 10.1109/TBDATA.2019.2921572.
- Hui, B., et al. (2024). Qwen2.5-Coder Technical Report. arXiv:2409.12186. https://huggingface.co/Qwen/Qwen2.5-Coder-0.5B-Instruct
- Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). C-Pack: Packaged Resources to Advance General Chinese Embedding (BGE embedding family). arXiv:2309.07597. https://huggingface.co/BAAI/bge-small-en-v1.5
- Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S., & Bossan, B. (2022). PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. Hugging Face. https://github.com/huggingface/peft
- Google DeepMind (2025). Gemini API Documentation (gemini-3.6-flash). https://ai.google.dev/gemini-api/docs